

AI RESUME SCREENING SYSTEM

Complete 3-Week Build Plan

Full-Stack Project with AI Integration

Java + Spring Boot + MySQL + Google Gemini AI + HTML/CSS/JS

Uma Nagamani Y

Time: 3 weeks at 3-4 hours/day | Difficulty: Medium | 2026 Trending Project

WHAT YOU'RE BUILDING

An AI-powered web application that helps HR teams quickly screen resumes against job descriptions. Solves a REAL problem that companies face every day.

How it Works (User Journey)

- HR Admin logs in to the system
- Posts a Job Description with required skills and experience
- Candidates can upload their resume (PDF format)
- OR HR uploads multiple resumes for a job
- AI analyzes each resume against the job description
- Returns a Match Score (0-100%)
- Highlights matching skills, missing skills
- Suggests interview questions based on candidate's profile
- HR sees a ranked list of candidates by score

Why This is a Standout Project

- AI integration = 2026 trending tech
- Solves a REAL business problem (every HR uses this)
- Combines Java + AI = unique skill combination
- Has clear value proposition (saves HR time)
- Easy to demo in interviews
- Stands out from typical CRUD projects

TECHNOLOGY STACK

Component	Technology + Why
Backend Language	Java 17 (LTS version, industry standard)
Backend Framework	Spring Boot 3.3.x (most popular Java framework)
Database	MySQL 8 (familiar from your training)
Database Mapping	Spring Data JPA + Hibernate
Authentication	Spring Security + JWT tokens
AI Service	Google Gemini API (FREE tier - 1500 requests/day)
PDF Processing	Apache PDFBox (extract text from PDF resumes)
File Upload	Spring MultipartFile
Frontend	HTML5 + CSS3 + Vanilla JavaScript (no framework)
API Type	REST APIs with JSON
Build Tool	Maven
Editor	IntelliJ IDEA Community (free)
Version Control	Git + GitHub
Deployment	Render.com (free tier)

WHY GOOGLE GEMINI AI? (Free Choice)

Comparison of AI Options:

AI Service	Free?	Setup Difficulty	Quality
Google Gemini	YES - 1500/day	Very Easy	Excellent
OpenAI ChatGPT	NO - paid	Easy	Excellent
Hugging Face	YES - limited	Medium	Good
Ollama (local)	YES - free	Hard	Good

WINNER: Google Gemini API

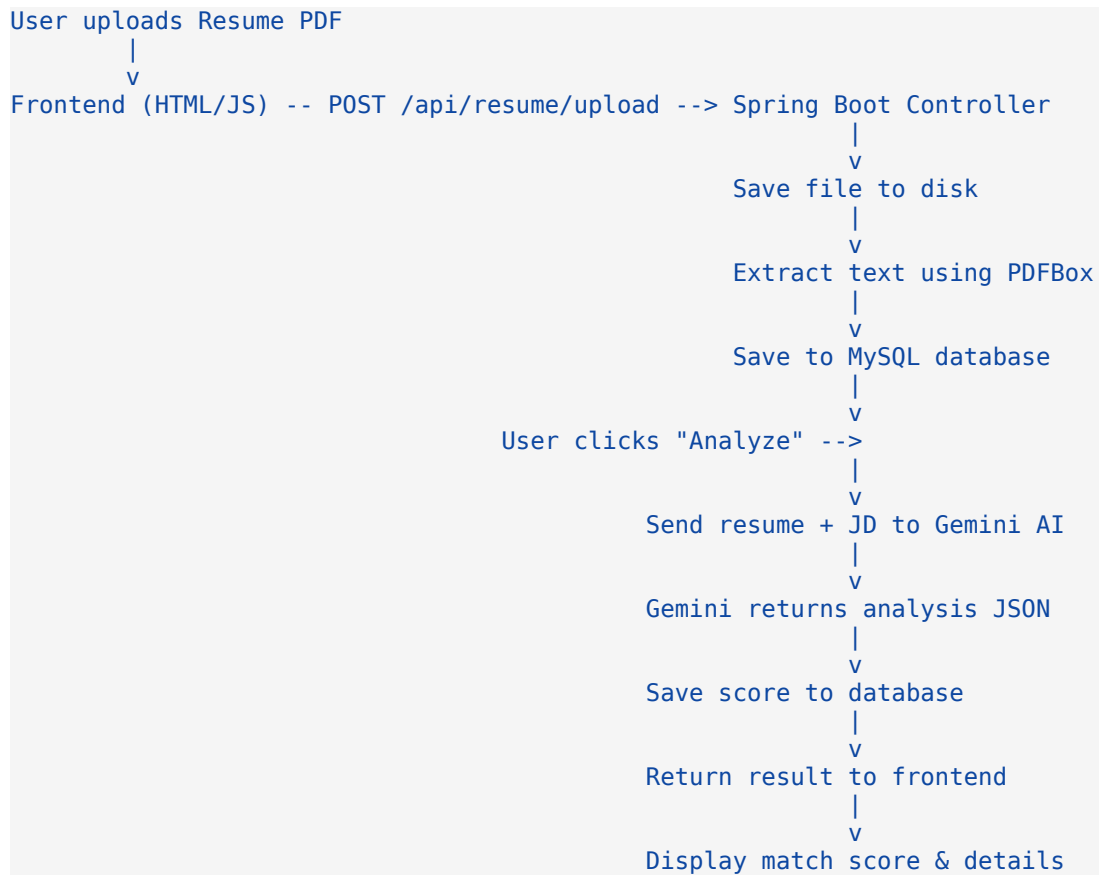
- Completely FREE for development
- 1500 requests per day (plenty for portfolio)
- Simple REST API integration
- Google-quality results
- No credit card required

How to Get Free Gemini API Key (5 minutes)

- Go to: <https://aistudio.google.com/app/apikey>
- Sign in with your Google account
- Click 'Create API key in new project'
- Copy the key (looks like: AlzaSy...)
- Save it somewhere safe
- That's it - ready to use!

HOW EVERYTHING CONNECTS

Here's how data flows through your system:



DATABASE TABLES (4 Tables)

Table 1: users

Stores HR Admin login info

```
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL, -- BCrypt encrypted
  role VARCHAR(20) DEFAULT 'HR',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Table 2: jobs

Stores job descriptions posted by HR

```
CREATE TABLE jobs (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  title VARCHAR(200) NOT NULL,
  description TEXT NOT NULL,
  required_skills TEXT,
  experience_required VARCHAR(50),
  location VARCHAR(100),
  salary_range VARCHAR(100),
  status VARCHAR(20) DEFAULT 'ACTIVE',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Table 3: resumes

Stores uploaded resume info

```
CREATE TABLE resumes (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  candidate_name VARCHAR(100) NOT NULL,
  candidate_email VARCHAR(100),
  file_name VARCHAR(255),
  file_path VARCHAR(500),
  extracted_text TEXT,
  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Table 4: screenings

Stores AI analysis results

```
CREATE TABLE screenings (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  job_id BIGINT NOT NULL,
  resume_id BIGINT NOT NULL,
  match_score INT, -- 0 to 100
  matching_skills TEXT,
  missing_skills TEXT,
  ai_summary TEXT,
  interview_questions TEXT,
  screened_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (job_id) REFERENCES jobs(id),
  FOREIGN KEY (resume_id) REFERENCES resumes(id)
);
```


REST API ENDPOINTS (12 Total)

Authentication APIs

Method	Endpoint	Purpose
POST	/api/auth/register	HR registers account
POST	/api/auth/login	HR login + get JWT

Job APIs

Method	Endpoint	Purpose
POST	/api/jobs	Create new job posting
GET	/api/jobs	Get all jobs
GET	/api/jobs/{id}	Get job by ID
DELETE	/api/jobs/{id}	Delete job

Resume APIs

Method	Endpoint	Purpose
POST	/api/resumes/upload	Upload resume PDF
GET	/api/resumes	Get all resumes
DELETE	/api/resumes/{id}	Delete resume

Screening APIs (The AI Magic!)

Method	Endpoint	Purpose
POST	/api/screening/analyze	AI analyzes resume vs job
GET	/api/screening/job/{jobId}	Get all candidates ranked
GET	/api/screening/{id}	Get screening details

WEEK 1: BACKEND FOUNDATION (Days 1-7)

Goal: Working Spring Boot API with database + authentication

Day 1: Project Setup (4 hours)

- Create Spring Boot project on start.spring.io
- Dependencies: Spring Web, Spring Data JPA, MySQL Driver, Validation, Lombok, Security
- Open in IntelliJ
- Create MySQL database: ai_resume_db
- Configure application.properties
- Get Gemini API key from aistudio.google.com
- Run project - verify 'Started Application' message

Day 2: User Entity + Authentication (4 hours)

- Create User entity with email, password, role
- Create UserRepository
- Setup Spring Security configuration
- Implement JWT utility class
- Create AuthController with /register and /login
- Test in Postman - get JWT token

Day 3: Job Entity + CRUD (4 hours)

- Create Job entity (title, description, skills, etc.)
- Create JobRepository
- Create JobService with business logic
- Create JobController with CRUD endpoints
- Add validation (@Valid, @NotBlank)
- Test all job endpoints in Postman

Day 4: Resume Entity + File Upload (5 hours)

- Create Resume entity
- Configure file upload limits in application.properties
- Create folder: uploads/resumes/
- Create ResumeController with /upload endpoint
- Save PDF to disk + path to database
- Test upload via Postman

Day 5: PDF Text Extraction (4 hours)

- Add Apache PDFBox dependency to pom.xml
- Create PdfService class
- Method: extractTextFromPdf(File pdfFile)
- Integrate with ResumeController

- Save extracted text to database
- Test with real PDF resumes

Day 6: Exception Handling + Validation (3 hours)

- Create GlobalExceptionHandler with @ControllerAdvice
- Custom exceptions: ResourceNotFoundException, BadRequestException
- Add @Valid annotations everywhere
- Test error scenarios in Postman

Day 7: Push to GitHub + Review (2 hours)

- Initialize Git repository
- Create .gitignore (exclude target/, .idea/, application.properties with API key)
- Commit and push to GitHub
- Create README.md (basic version)
- Review all code, refactor if needed

WEEK 1 MILESTONE: Working Spring Boot API with auth, jobs, resumes, PDF extraction. Tested in Postman. Code on GitHub.

WEEK 2: AI INTEGRATION + SCREENING LOGIC (Days 8-14)

Goal: Working AI analysis - the heart of your project

Day 8: Gemini API Setup (3 hours)

- Add Gemini SDK or use RestTemplate to call REST API
- Add API key to application.properties (use environment variable!)
- Create GeminiService class
- Test simple prompt: 'Hello, are you working?'
- Print response in console

Day 9-10: AI Prompt Engineering (6 hours total)

- Design the prompt to send to Gemini
- Prompt should ask for: match score, matching skills, missing skills, summary, interview questions
- Use structured JSON output format
- Test prompt manually in Gemini playground first
- Fine-tune until responses are consistent

Day 11: Screening Entity + Repository (3 hours)

- Create Screening entity (job_id, resume_id, score, etc.)
- Create ScreeningRepository
- Add custom queries (find by job, sort by score)

Day 12: Screening Service Logic (5 hours)

- Create ScreeningService class
- Method: analyzeResume(jobId, resumeId)
- Fetch job and resume from DB
- Send to Gemini API with prompt
- Parse JSON response from Gemini
- Save screening results to DB

Day 13: Screening Controller + Testing (4 hours)

- Create ScreeningController
- POST /api/screening/analyze endpoint
- GET /api/screening/job/{jobId} - candidates ranked by score
- Test full flow in Postman:
 - 1. Create job
 - 2. Upload resume
 - 3. Analyze (calls Gemini)
 - 4. View results

Day 14: Polish Backend + Push (3 hours)

- Add logging (Spring Boot Logger)

- Add response DTOs (don't expose entities directly)
- Code review and refactor
- Update README
- Push to GitHub

WEEK 2 MILESTONE: AI integration WORKING. Resumes are analyzed by Gemini. Real match scores returned.

WEEK 3: FRONTEND + DEPLOYMENT (Days 15-21)

Goal: Beautiful HTML/CSS/JS frontend + deployed live

Day 15: Frontend Project Structure (3 hours)

- Create folder: src/main/resources/static/
- Create files: index.html, dashboard.html, css/style.css
- Create JS files: js/api.js, js/auth.js, js/dashboard.js
- Add basic boilerplate HTML5 structure
- Test by opening in browser via http://localhost:8080/

Day 16: Login & Register Pages (4 hours)

- Build login.html with email + password form
- Build register.html with name + email + password
- Style with CSS (use a nice color scheme)
- JS: On form submit, call /api/auth/login or /register
- Store JWT token in localStorage
- Redirect to dashboard.html on success

Day 17: Dashboard - Job Section (5 hours)

- dashboard.html with tabs: Jobs / Resumes / Screening
- Jobs tab: List all jobs in cards
- 'Create Job' button opens form modal
- JS: Fetch /api/jobs on page load
- JS: POST new job, refresh list

Day 18: Dashboard - Resume Upload (4 hours)

- Resumes tab: List uploaded resumes
- Upload form with file picker (PDF only)
- JS: Upload file using FormData and fetch API
- Show upload progress
- Refresh list after upload

Day 19: Dashboard - Screening (THE STAR FEATURE!) (5 hours)

- Screening tab: Select Job + Select Resume + 'Analyze' button
- Show loading spinner during AI analysis (takes 5-10 sec)
- Display result beautifully:
 - - Big match score with color (green/yellow/red)
 - - Matching skills (green badges)
 - - Missing skills (red badges)
 - - AI summary text
 - - Interview questions list

- 'Candidates' view: List all candidates for a job, sorted by score

Day 20: Polish + Mobile Responsive (4 hours)

- Add nice animations (CSS transitions)
- Make all pages mobile-responsive
- Add logout button
- Add error messages (toast notifications)
- Test full user flow end-to-end
- Fix any bugs found

Day 21: Deploy + Documentation (4 hours)

- Create account on Render.com
- Push final code to GitHub
- Setup PostgreSQL on Render (or use Railway for MySQL)
- Create web service on Render
- Add environment variables (DB password, JWT secret, Gemini API key)
- Deploy! Get live URL
- Create comprehensive README with screenshots
- Record 2-min demo video for LinkedIn

WEEK 3 MILESTONE: LIVE deployed app with public URL. Ready to share with recruiters!

KEY CODE SAMPLES

1. application.properties Configuration

```
spring.application.name=ai-resume-screening
server.port=8080

# Database
spring.datasource.url=jdbc:mysql://localhost:3306/ai_resume_db?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# File Upload
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
file.upload-dir=./uploads/resumes/

# JWT
jwt.secret=${JWT_SECRET}
jwt.expiration=86400000

# Gemini AI
gemini.api.key=${GEMINI_API_KEY}
gemini.api.url=https://generativelanguage.googleapis.com/v1beta/models/gemini-pro:generateContent
```

2. Job Entity (Sample)

```
@Entity
@Table(name = "jobs")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Job {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Title is required")
    private String title;

    @NotBlank(message = "Description is required")
    @Column(columnDefinition = "TEXT")
    private String description;

    @Column(name = "required_skills", columnDefinition = "TEXT")
    private String requiredSkills;

    private String experienceRequired;
    private String location;
    private String salaryRange;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }
}
```

3. Gemini AI Service (THE MAGIC!)

```
@Service
public class GeminiService {

    @Value("${gemini.api.key}")
    private String apiKey;

    @Value("${gemini.api.url}")
    private String apiUrl;

    private final RestTemplate restTemplate = new RestTemplate();

    public ScreeningResult analyzeResume(String jobDescription, String resumeText) {
        // Build the prompt
        String prompt = buildAnalysisPrompt(jobDescription, resumeText);

        // Call Gemini API
        String response = callGeminiApi(prompt);

        // Parse response and return result
        return parseGeminiResponse(response);
    }

    private String buildAnalysisPrompt(String jd, String resume) {
        return """
        Analyze this resume against the job description.
        Return ONLY valid JSON in this format:
        {
            "matchScore": 85,
            "matchingSkills": ["Java", "Spring Boot"],
            "missingSkills": ["AWS", "Docker"],
            "summary": "Strong candidate with...",
            "interviewQuestions": ["Q1", "Q2", "Q3"]
        }

        JOB DESCRIPTION:
        %s

        RESUME:
        %s
        """.formatted(jd, resume);
    }

    private String callGeminiApi(String prompt) {
        // Build request body
        Map<String, Object> requestBody = Map.of(
            "contents", List.of(Map.of(
                "parts", List.of(Map.of("text", prompt))
            ))
        );

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);

        HttpEntity<Map<String, Object>> entity =
            new HttpEntity<>(requestBody, headers);

        String url = apiUrl + "?key=" + apiKey;

        ResponseEntity<String> response =
            restTemplate.postForEntity(url, entity, String.class);

        return response.getBody();
    }
}
```



```
}  
}
```

HOW TO PRESENT THIS ON YOUR RESUME

Resume Bullet Points:

AI Resume Screening System (Full-Stack Web Application)

Tech: Java 17 | Spring Boot 3 | MySQL | Google Gemini AI | JWT | HTML/CSS/JS | Apache PDFBox

- Designed and built a full-stack AI-powered web application that helps HR teams screen resumes against job descriptions using Google Gemini AI, reducing manual screening time by an estimated 80%
- Implemented secure REST APIs with Spring Boot 3 and JWT authentication, supporting 12 endpoints for job management, resume upload, and AI-powered analysis
- Integrated Google Gemini API for resume-to-job matching with custom prompt engineering, returning structured JSON responses with match scores, skill gaps, and interview question suggestions
- Built PDF parsing pipeline using Apache PDFBox to extract text from uploaded resume files for AI analysis
- Developed responsive frontend with vanilla HTML, CSS, and JavaScript that consumes backend APIs and provides intuitive UX for HR users
- Deployed live on Render.com with MySQL database, available at [your-url]

LinkedIn Post Template:

 Just deployed my AI-Powered Resume Screening System!

A full-stack web application that helps HR teams screen resumes against job descriptions using Google's Gemini AI.

 Live Demo: [your-url]

 GitHub: [your-github-url]

✨ Features:

- HR Admin login with JWT authentication
- Job posting with required skills
- PDF resume upload with automatic text extraction
- AI-powered match scoring (0-100%)
- Skill gap analysis (matching + missing skills)
- AI-generated interview questions

🔧 Tech Stack:

- Backend: Java 17, Spring Boot 3, MySQL
- AI: Google Gemini API
- PDF Processing: Apache PDFBox
- Frontend: HTML, CSS, JavaScript
- Security: Spring Security + JWT
- Deployment: Render.com

This project showed me the power of integrating AI into real-world business applications. The same screening that takes humans 30 minutes is done in 10 seconds.

#JavaDeveloper #SpringBoot #AI #SoftwareEngineering #OpenToWork

COMMON ISSUES & SOLUTIONS

Problem: Gemini API returns empty response

- Check API key is correct
- Check you haven't exceeded rate limit (15/minute)
- Print the full request body to debug
- Try simpler prompt first

Problem: PDF extraction returns garbled text

- Some PDFs are images (scanned) - PDFBox can't extract text
- Use only text-based PDFs for testing
- For production, would need OCR (Tesseract)

Problem: File upload fails with 'request too large'

- Increase limits in application.properties
- Check both max-file-size and max-request-size

Problem: AI returns inconsistent JSON

- Make prompt VERY explicit about JSON format
- Add 'Return ONLY valid JSON, no markdown' to prompt
- Add try-catch when parsing JSON response

Problem: Deployment fails on Render

- Check Render build logs
- Verify environment variables are set
- Make sure pom.xml has Spring Boot Maven plugin

FINAL NOTES

Uma — this is a REAL 2026-grade project. When you finish this:

- Your resume will stand out from 90% of candidates
- Recruiters will WANT to interview you to learn about it
- You'll have AI skills that command higher salaries
- You can talk about REAL business problems, not academic exercises

REALISTIC EXPECTATIONS:

- Week 1 will feel familiar (Spring Boot you know)
- Week 2 will feel exciting and hard (AI integration is new!)
- Week 3 will feel like assembling the puzzle pieces
- Expect 1-2 days of being STUCK on something - that's normal
- Google + Stack Overflow are your best friends

If you finish even 70% of this project — it's still impressive on your resume.

Start TODAY. Day 1: Project setup. Just 3 hours. Don't wait for Monday.

You have time now (since interview was cancelled). USE IT.

You've got this, Uma. 🚀